Sankhya

An LP / MILP / QP solver built from scratch — design decisions, what is tested and how, where it stands, and what happens next

30 August 2026

Contents

0. What this document is

A full technical account of the project: what is being built, every decision that shaped it and what was rejected instead, what has been measured and how, where it honestly stands against real solvers, and what the remaining work is.

It assumes a computer science background — complexity, data structures, memory layout, testing, parallelism — and assumes **no** background in mathematical optimization. Part I builds that from scratch, at the level of someone who knows what NP-hard means but has never written a simplex.

Nothing here is aspirational. Every number has the command that produces it, and where something does not work, it says so.

Part I — The problem

1. The brief

Smart India Hackathon 2026, problem statement **SIH26119**, proposed by MRPL: an **indigenous, GPU-accelerated optimization solver** for refinery planning.

Three words in that carry all the constraints.

"Optimization solver" — the deliverable is a solver, not an application that calls one. Input is a model file; output is an optimal decision vector plus a proof that it is optimal.

"Indigenous" — do not wrap Gurobi, CPLEX, HiGHS, or SciPy. The algorithms have to be implemented. This is the constraint that makes the project large: an LP solver is roughly the same order of work as a small database engine.

"GPU-accelerated" — the hardware has to actually be used, and this constrains the *choice of algorithm* far more than it constrains the code. Most of classical optimization is sequential by nature. Section 6 is about that.

2. What a refinery planning problem actually is

A refinery buys crude oils, runs them through processing units, and blends the resulting streams into finished products. The decisions are quantities: how many barrels of each crude to buy, how much of each intermediate stream to route to which unit, how much of each stream goes into each product.

The constraints are:

- **Capacity.** Each unit processes at most so much per day.
- **Material balance.** What enters a unit leaves it, as products of known yields. Nothing appears or disappears.
- **Specification.** Each finished product must meet limits — sulphur, octane, density. These are weighted averages over what went into the blend.
- **Demand and inventory.** Sell no more than the market takes; carry the rest.

The objective is margin: revenue minus crude cost minus processing cost.

Written down, this is a **linear program**. The refining industry has known this since the 1950s; what has changed is scale and turnaround. A real planning model has tens of thousands of constraints and planners want to re-run it many times a day as crude prices move.

The repository contains such a model at `data/refinery/refinery.mps`, generated by `scripts/refinery_model.py` — twelve periods, eight crudes, CDU with fixed cut yields, hydrotreater and FCC, linear blending with sulphur specifications, and inventory carried between periods. It exists because every public benchmark instance available is a $0/\pm 1$ combinatorial LP, and none of them has the property that matters here: a coefficient dynamic range of about 4×10^5, because barrels, fractions and rupees appear in the same matrix.

3. The three problem classes

3.1 Linear program

$$\min_x \; c^\top x \quad \text{s.t.} \quad l \le Ax \le u, \quad lo \le x \le hi$$

$x \in \mathbb{R}^n$ are the decisions, $A \in \mathbb{R}^{m \times n}$ is sparse (a constraint mentions a handful of variables out of thousands).

Why it is tractable. The feasible region is an intersection of half-spaces — a convex polyhedron. A linear objective's level sets are parallel hyperplanes. Slide one outward until it last touches the region, and the last contact is at a **vertex**. So there is always an optimal solution at a corner, and the search space collapses from a continuum to a finite set.

Why that is not enough. A vertex is where n constraints hold with equality, so there are up to $\binom{m}{n}$ of them. For $m = 200$, $n = 100$ that exceeds the number of atoms in the observable universe. You cannot enumerate; you have to walk.

3.2 Mixed-integer program

Add $x_j \in \mathbb{Z}$ for some subset of variables. Needed the moment a decision is discrete: run this unit or not, buy this cargo or not, use this recipe or not. Also used to approximate nonlinear behaviour piecewise.

The convexity argument dies. The feasible set becomes a lattice of points inside a polyhedron, and the optimal integer point need not be near any vertex of the relaxation. **MILP is NP-hard.**

A concrete example, because "just round the LP solution" is what everyone tries first:

$$\max\ 5x_1 + 4x_2 \quad \text{s.t.} \quad 4x_1 + 5x_2 \le 20,\ 3x_1 + 2x_2 \le 12,\ x \ge 0 \text{ integer}$$

The LP relaxation gives $x = (2.857, 1.714)$, objective 21.14. Round it and $(3, 2)$ violates both constraints; $(3, 1)$ gives 19; $(2, 2)$ gives 18. The true integer optimum is $(4, 0)$ with objective **20** — nowhere near the fractional point.

3.3 Quadratic program

$$\min_x\ \tfrac{1}{2} x^\top Q x + c^\top x \quad \text{s.t.} \quad l \le Ax \le u$$

A quadratic objective is what you write to penalise deviation: "stay close to last month's plan" is $\min \|x - x_{\text{prev}}\|^2$. Any least-squares fit or risk term is quadratic. Still convex when $Q \succeq 0$, so still tractable — but it needs a different algorithm.

4. Duality — the one concept everything else depends on

This is worth doing properly because the simplex's termination test, branch-and-bound's pruning rule, shadow prices, and half of presolve are all this one idea.

4.1 The construction

Take a maximisation with $Ax \le b$, $x \ge 0$. How would you prove an upper bound on the optimum **without searching**?

Pick non-negative weights y and add up the weighted constraints: $y^\top Ax \le y^\top b$. If the weighted coefficients dominate the objective — $A^\top y \ge c$ componentwise — then for any feasible $x \ge 0$:

$$c^\top x \;\le\; (A^\top y)^\top x \;=\; y^\top (Ax) \;\le\; y^\top b$$

So $b^\top y$ is an upper bound, and producing y *is* the proof. The tightest such bound is itself an LP:

$$\min_{y} \; b^\top y \quad \text{s.t.} \quad A^\top y \ge c,\; y \ge 0$$

That is the **dual**. Structurally: one dual variable per primal constraint, one dual constraint per primal variable, the matrix transposed, the objective and right-hand side exchanged.

4.2 The two theorems

Weak duality. $c^\top x \le b^\top y$ for every feasible pair. The derivation above, no assumptions.

Strong duality. At the optimum they are **equal**. The best bound is exactly the answer.

Strong duality is what makes "optimal" a checkable claim rather than an assertion. The solver returns x and y, both feasible, with matching objectives — and anyone can verify that in two matrix-vector products without trusting the solver at all.

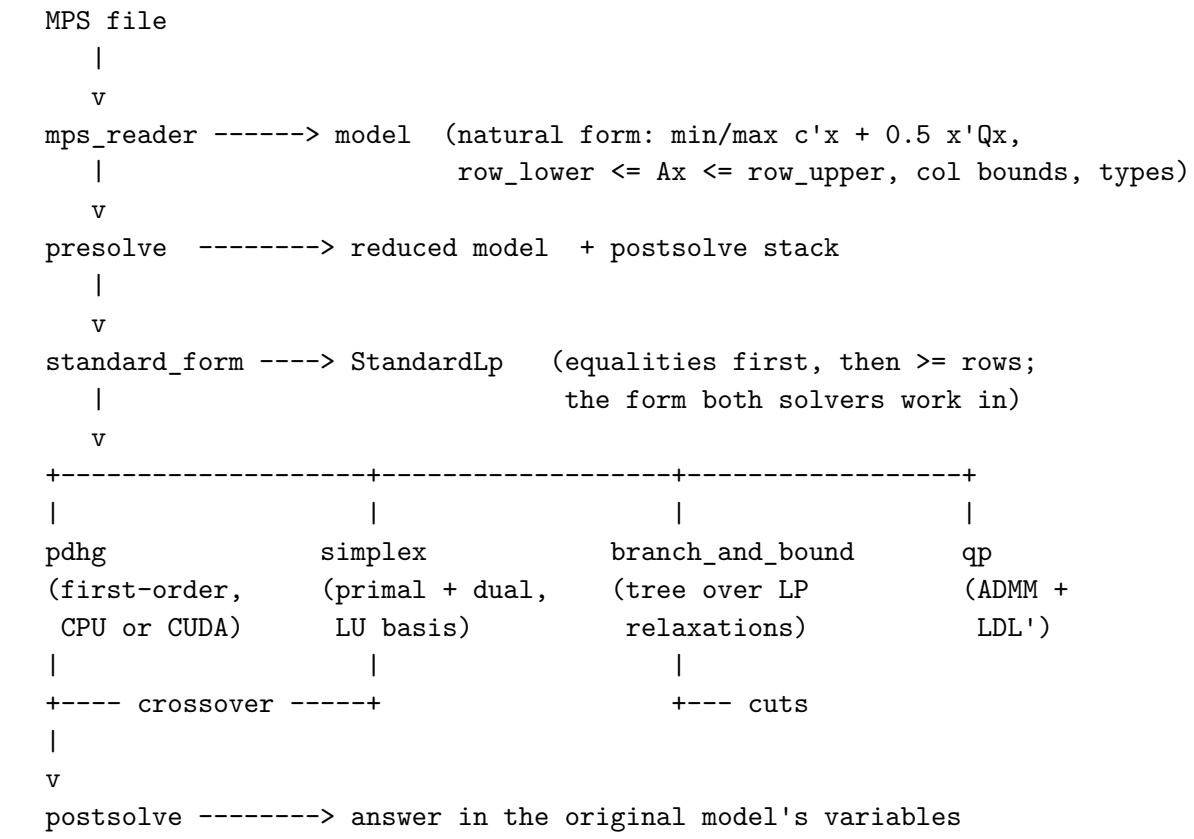
4.3 Reduced costs and shadow prices

The **reduced cost** of variable j is $d_j = c_j - (A^\top y)_j$: its own cost minus what the constraints pay for using it. At an optimum, **complementary slackness** holds — a constraint is either tight or its dual is zero, and a variable is either strictly between its bounds or its reduced cost may be nonzero.

The dual value y_i is the **shadow price** of constraint i: the marginal objective gain per unit of that resource. For a refinery that is a direct answer to "which unit should we debottleneck", and it falls out of the same solve for free. This is why the code goes to real trouble to recover duals through presolve, and flags the cases where recovery is only approximate — a shadow price someone might size capital on has to be trustworthy or absent.

Part II — Architecture and decisions

5. Component map



Source sizes, as a sense of where the mass is:

file	lines	what
`simplex.cpp`	1,283	primal and dual, basis, pricing, ratio test
`branch_and_bound.cpp`	1,247	the MILP tree, heuristics, node management
`presolve.cpp`	1,226	eleven reductions plus postsolve
`pdhg.cpp`	1,090	the first-order method, restarts, polishing
`mps_reader.cpp`	728	the file format
`cuda_backend.cu`	623	device kernels
`qp.cpp`	608	ADMM
`cuts.cpp`	538	cover, MIR, Gomory separation
`lu.cpp`	318	Markowitz LU with product-form updates
`crossover.cpp`	~190	first-order point → simplex basis
`ldl.cpp`	144	sparse $LDL^\top$ for the QP's KKT system

C++17, no external dependencies beyond the standard library. CMake. CUDA is optional and compiled out when absent.

6. The central design decision: which algorithms

There are three families of LP algorithm and the choice determines everything downstream.

	mechanism	strength	weakness
Simplex	walk vertex to vertex along edges	exact; warm starts in a few pivots	inherently sequential
Interior point	cut through the interior, Newton steps	strong on large problems	sparse factorisation per iteration; no warm start
First-order	gradient step + projection	only matrix-vector products; parallel	slow to high accuracy

Two were implemented: simplex and first-order. The reasoning, in full, because "why not interior point" is the first question anyone with a numerical background asks.

6.1 Why first-order

Its entire iteration is:

$$x^{k+1} = \Pi_{[lo,hi]}(x^k - \tau(c - K^\top y^k)), \qquad y^{k+1} = \Pi_{y\geq 0}(y^k + \sigma(q - K(2x^{k+1} - x^k)))$$

Two sparse matrix-vector products, some vector arithmetic, two clamps. No factorisation, no basis, no sequential bookkeeping. An SpMV is thousands of independent row dot-products; a clamp is elementwise. **This is the only LP algorithm whose inner loop is natively parallel**, which is why every GPU LP solver in the literature (PDLP, cuPDLP, cuPDLPx, HPR-LP) is in this family.

The problem statement asks for GPU acceleration. This is the algorithm that answers it.

6.2 Why simplex as well

First-order methods converge quickly to moderate accuracy and slowly to high accuracy. For a refinery that is the wrong shape of error — a plan that violates a unit capacity by 0.8 barrels cannot be run even if its objective looks right.

More importantly, **MILP needs warm starts.** Branch-and-bound solves one LP per node and the nodes differ by a single changed bound. Since a reduced cost does not depend on a variable's bound, the parent's optimal basis is still *dual* feasible for the child — so the dual simplex repairs it in a handful of pivots. Measured here: **18,772 of 18,775 node relaxations warm start, at roughly three pivots each.** A cold solve per node would be hundreds. Branch-and-bound is only affordable because of this.

6.3 Why not interior point

Against, mechanically. An IPM factorises $ADA^\top$ (or an augmented KKT system) at every iteration, with a fill-reducing ordering. Large machinery that nothing else in the codebase reuses.

Against, structurally. It serves neither goal. It does not warm start, so branch-and-bound would collapse; and its cost centre is a sparse factorisation, which is the part that parallelises worst.

For, and this is real. IPMs converge to high accuracy natively. That is precisely where first-order methods are weakest — it is why `--gap-tol` exists and why feasibility polishing had to be built.

And the ground has moved. NVIDIA shipped cuDSS (GPU sparse Cholesky/$LDL^\top$/LU), and condensed-space IPM formulations reshape the KKT system into something that factorises far better on a device.

The decision still holds, for a different reason than originally given. Not "IPM cannot work on a GPU" — that is less true every year — but:

1. The accuracy an IPM adds is already covered by the simplex.
2. Reported gains for fully GPU-based interior-point LP remain modest; the sparse factorisation is still the bottleneck.
3. It still would not warm start for the tree.

The honest concession is QP. Commercial solvers use a barrier method for QP, and the five QP instances that fail here (Section 13.4) fail on step-size tuning, which an IPM has no equivalent of. *"An IPM would be better for QP"* is true. It was not built because QP is the smallest of the three components and the machinery is three to four weeks.

This is written down so the decision can be revisited on evidence rather than re-argued from memory.

7. What the first-order method actually is

Worth stating precisely because it is checkable.

The implementation is PDHG with restarts, Halpern anchoring, and reflection:

$$z^{k+1} = \tfrac{k+1}{k+2}\, T(z^k) + \tfrac{1}{k+2}\, z^0, \qquad R(z) = T(z) + \gamma\,(T(z) - z)$$

where T is one PDHG step. Chen, Sun, Yuan, Zhang and Zhao (arXiv:2509.23903), **Proposition 3.1**: reflected restarted Halpern PDHG at $\gamma = 1$ **is** the Halpern Peaceman–Rachford method, with the semi-proximal term $T_1 = \lambda_A I - AA^*$ and $\lambda_A = 1/\eta^2 \geq \|A\|^2$.

Our default reflection is 1.0. So this is an **HPR method**, not a pile of heuristics on top of PDHG.

The same paper measures the gap to the best implementation, HPR-LP, at 10^{-8}: on the Mittelmann LP set both solve 44 of 49 with HPR-LP about 1.1× faster; on MIPLIB relaxations HPR-LP solves two more and is 1.8× faster. Their conclusion is that both are effective realisations of the same method and the difference is **implementation, not algorithm**. That is the useful framing: the remaining gap to the frontier is engineering, not a missing idea.

8. Feasibility polishing

First-order methods trade feasibility against optimality along their path. PDLP's observation (arXiv:2501.07018 §4) is that a *pure feasibility* problem — same constraints, zero objective — is far easier for the method, because nothing pulls against the constraints. And PDHG's iterates are non-increasing in distance to any optimal solution, so a run warm started near a good point stays near it.

So once the duality gap is acceptable, pause and solve two subproblems warm started from the current point:

$$\textbf{primal: } c := 0 \text{ from } (x, 0), \qquad \textbf{dual: } q := 0, \text{ finite bounds} := 0 \text{ from } (0, y)$$

On the refinery model at shipped defaults:

	without	with
iterations	160,720	**12,800** + 1,720
capacity violation	1.46e-02	**1.28e-04**
duality gap	1.1e-09	4.5e-03

Eleven times fewer iterations for a violation 114× smaller, at the cost of a 0.45% gap. For a planning model that is the right trade: a plan that overruns a unit is not a plan; a plan leaving half a percent on the table is.

9. Crossover — connecting the two solvers

The two LP solvers work on the same `StandardLp` and, for most of the project, never interacted. They fail in opposite ways: the first-order method reaches a *point* quickly and a vertex never; the simplex reaches a *vertex* with a certificate and pays for the walk.

Crossover is the handoff. Solve loosely, read off which columns that point wants basic, and start the simplex from that basis rather than from all slacks.

Reading a basis off a point. A column strictly inside its bounds cannot be sitting *at* one, so it must be basic. Score each column by distance from its nearest bound relative to its own range, take them best-first. Since a basic column also prices at zero at an optimum, the reduced cost is folded into the ranking (Section 13.2 has the sweep that set its weight).

The trap. The obvious implementation picks the top m columns and factorises. That fails: nothing guarantees m columns chosen this way are linearly independent, and an earlier attempt in this codebase produced singular starting bases on `scfxm1`, `bandm` and `degen2`.

So it is done the other way round. Start from the all-slack basis, which is $-I$ and nonsingular by inspection, and **pivot candidates in one at a time** through the same update the simplex uses. A pivot on a nonzero element maps a nonsingular basis to a nonsingular one, so non-singularity is an *invariant*, not a hope.

Two further traps, both found on `cycle`. Pushing 749 columns without ever refactorising decayed the product form until the answer was wrong. Refactorising periodically fixed that — and the basis

handed over still only worked *through its accumulated updates*, so the simplex factorised it from scratch, rejected it, and started cold while the run reported success. Both are handled by keeping the last basis that factorised from scratch and handing that one over.

And the safety rule: if the seeded solve does not reach an optimum, the cold solve is run and that is the answer. Crossover can save pivots or do nothing; it cannot cost an answer.

Part III — How this is tested

This part matters more than any speed number, and it is the part most solver write-ups skip. A slow solver announces itself. A **wrong** solver returns a confident number with a matching dual bound and no complaint — and this one has done exactly that, repeatedly. Everything below exists because something got through.

10. The testing pyramid

Layer 1 — unit tests

Twelve suites, no external framework (the point of the project is that the stack is its own; a test runner is not worth an exception):

```
test_sparse   test_mps      test_standard_form   test_scaling
test_pdhg     test_backend  test_cuts            test_lu
test_simplex  test_crossover test_presolve       test_ldl
```

Run with:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j8
for t in build/tests/test_*; do "$t"; done
```

The design rule that matters here: a test must never validate a component using a quantity that component computed. Two examples of the rule and one of what happens without it.

- The $LDL^\top$ test builds K, picks an x, forms $b = Kx$, solves, and compares against the x it started from. It never checks a residual the factorisation computed about itself. This rule exists because the factorisation once silently degenerated into a *diagonal* one — it had been handed the upper triangle where it needs the lower, so every off-diagonal entry was skipped and the elimination tree came out empty. The self-residual test passed at 10^{-14} and said nothing.
- The cut tests enumerate all integer points of small programs and check that no feasible point is cut off — and separate at **simplex vertices** as well as at random interior points. That last detail found a real bug: the cover separator inferred whether an item was complemented by testing `slack == x_j`, which is true for complemented items and false otherwise — *except* at exactly $x_j = 0.5$, where $1 - x_j$ is also 0.5. 426 separations at random points never landed there. A simplex vertex sits a binary at exactly 0.5 routinely.
- Three existing dual-recovery tests all passed all-zero reduced costs, which is the one input that cannot distinguish a correct implementation from a broken one. They did not catch an index-space bug where a reduced-cost array indexed by *reduced* column was being read with *original* column indices.

Layer 2 — verification against published optima

Unit tests check components. This checks answers.

```
python3 -u bench/verify_simplex.py 60      # all 88 Netlib, vs published optima
python3 -u bench/miplib_survey.py 15       # 103 MIPLIB, vs published optima
```

Both compare the returned objective against a published value and shout when a run claims `optimal` and does not match. This is the layer that catches the class of bug unit tests structurally cannot: a

component that is individually correct in a combination that is not.

The `fiber` case is the canonical example. The MILP returned 652,748.78 against a true 405,935.18 — **60.8% wrong**, proved optimal, matching dual bound, no complaint. Two independent causes, and *each cut family was valid on its own*:

1. A bound crossing of 1.78×10^{-15} — the last bit of a double — was treated as proof that a subproblem was empty, while the bound-propagation routine tolerates crossings up to 10^{-7}. **The two disagreed about what an empty box is**, and the answer fell through the gap.
2. A cover cut was derived from a previously-generated *cut row* rather than a model row.

Only the combined run reached the point that produced the bad cut. No unit test would have found this.

Layer 3 — cross-validation against an independent solver

HiGHS is installed and used as a third opinion. This is not decoration — it settled the most confusing episode in the project's history.

A full-set verification made the solver look wrong on **eleven** Netlib instances at once. Eleven simultaneous disagreements is usually the reference and not the code, and it was: HiGHS returns what this solver returns on **eight** of them. `e226`'s stored value was off by exactly 7.113, which is that instance's own objective constant — the classic Netlib table discrepancy over whether the objective row's constant is included.

The reference file was corrected, with a note recording why. The other three were genuine and are fixed (Section 12).

Layer 4 — self-checks inside the solver

Tests run when someone runs them. These run always.

Dual feasibility check. The simplex verifies its own optimality conditions before claiming optimality. It found a wrong answer on its first run: the dual simplex returned `fit1p` at 33,609 against a true 9,146.38 — feasible, row violation 2.8×10^{-14}, reported optimal. Cause: Bland's anti-cycling rule was overriding the *dual* ratio test. In the primal, pricing and the ratio test are separate steps, so overriding pricing is safe; in the dual the ratio test **is** the entering choice and the only thing keeping reduced costs correctly signed. Removing the override took the suite from 13/16 to 16/16.

Recompute-from-the-matrix feasibility check. Before the word "optimal" leaves the simplex, Ax is recomputed **from the matrix**, not read off the basis, and the bounds are checked too.

This one is worth dwelling on, because it is the structural lesson of the whole project:

> Every check the solver runs goes through the same basis. When the factorisation decays, the computed x_B is wrong — and the optimality test, the feasibility test and the residual all ask that same decayed representation and get consistent answers. **A solver cannot audit itself through the object that is broken.**

The CLI repeats the check against the *original* model after postsolve, because with presolve the simplex is solving a different problem and a point satisfying the reduced rows can still miss the original ones.

Cut validity net. A valid cut can only raise a bound. If the root bound falls after a round of cuts, at least one cut was invalid — the round is rolled back. This does not catch a bad cut that happens to raise the bound (nothing cheap does), so it is a net, not a proof.

Layer 5 — the debug-solution tracker

`BranchAndBoundOptions::debug_solution`. Hand the tree a solution known to be optimal. Every point at which a node could be discarded first checks whether that solution lies inside the node's box, and the **first prune that throws it away names itself**.

A branch-and-bound that returns a wrong answer has pruned the optimum, and there is no way to reason backwards from a wrong answer to the prune that caused it. This turns that into a printed line. SCIP carries the same facility for the same reason. It found both `fiber` bugs in minutes after four layers of manual elimination had found neither.

Layer 6 — invariant checks that do not depend on a reference

- `bench/verify_presolve.py` — presolving must not change any answer, primal or dual.
- `bench/verify_reader.py` — the reader must agree with HiGHS on all 88 Netlib models.
- `bench/ablation.py` — every optional feature on and off, so a default that has stopped paying is visible.

11. How measurement is done, and three ways it goes wrong

A benchmark under load is not a benchmark. This has produced a false regression twice — once on `mas76` under adaptive cuts, once on `gen-ip054` under propagation. Both changes were pure improvements; re-running alone showed identical results. Had either been believed, a real improvement would have been discarded. Iteration and node counts are load-independent and are therefore preferred over wall-clock wherever the question allows.

`zsh` does not word-split unquoted variables. A variable holding two flags arrives as a single argument, the program rejects it, and the harness silently reads a stale result file. Cost three separate debugging sessions despite being documented.

A binary linked against a static library does not relink when the library is rebuilt. Twenty minutes spent reading correct code that was not being executed.

And a rule about attribution. Change one thing at a time. The most recent instance: the ratio-test fix and a crossover ranking change landed close together, and the crossover ratio moved from $0.51\times$ to $0.77\times$ on the full set with no way to say which caused it. That is currently being re-measured with one variable held fixed, which is the only way to find out.

Part IV — Where this stands

Every number below has the command that produces it. Machine: Apple Silicon arm64, macOS 26.5, Apple Clang 21.0, Release build. GPU numbers are from a rented NVIDIA Tesla T4.

12. Correctness

All 88 Netlib instances with published optima, cold, 60 s each:

	count
reach the published optimum	**77–78**
return a wrong answer	**0**
stop visibly (time / iteration limit / numerical error)	10–11

```
python3 -u bench/verify_simplex.py 60
```

The range is because two instances sit near the time limit and move depending on machine load.

This replaces a "16 of 16" figure that stood in the results document until 2026-08-30. That number was true of a sixteen-instance subset and it was **hiding six wrong answers** on the rest:

instance	reported	true
grow15	−205,842,493	−106,870,941
grow22	−68,986,355,650	−160,834,336
maros	−102,064.67	−58,063.74
modszk1	"unbounded"	320.61972906
scsd1	"unbounded"	8.6666666743
cycle	−30.888	−5.2263930249

The first three reported `optimal` at points missing the rows by up to 1.8×10^9.

One cause. The primal ratio test broke ties by row, and under Bland's rule by index, but **never by the size of the pivot**. So a candidate winning the ratio by 10^{-12} and losing the pivot magnitude by six orders of magnitude was taken; the basis update divides by that pivot; the factorisation decays; x_B stops meaning anything. The project's own design notes described the rule as implemented. It was not.

The fix is deliberately narrow — it fires only when the pivot about to be taken is below 10^{-2} of the largest entry in its own column, because an unconditional preference for the larger pivot loses `blend`. The threshold was swept, not chosen: 10^{-4} and 10^{-3} give 12 of 16 on the probe set, 10^{-2} and 10^{-1} give 13, and 0.3 gives 11.

Behind it sit the two recompute-from-the-matrix guards of Section 10. `cycle`, `modszk1` and `scsd8` now fail **visibly** rather than quietly, which is the correct outcome for an instance the method genuinely cannot handle.

13. Component results

13.1 Reader

Matches HiGHS on all 88 Netlib models, 1.4–1.5× faster. Free-format and fixed-format MPS, RANGES, BOUNDS, integer markers, quadratic sections.

One gap: `INDICATORS` sections (indicator constraints) are rejected with a clear message rather than mis-parsed. One MIPLIB instance out of 103 uses them.

13.2 Simplex and crossover

Primal and dual, Markowitz LU with product-form updates, Devex pricing, piecewise-linear phase one, incremental reduced-cost updates.

Incremental pricing. The primal used to make two full passes over the matrix per iteration — `compute_duals` recomputing every reduced cost, then the Devex weight update making a pass of the same shape over the pivot row. The pivot row α_{rj} is exactly what an incremental update needs, so the two are now one pass:

	recompute	incremental
iterations	154,739	**126,502** (0.818×)
time	99.1 s	**79.4 s** (0.801×)

Crossover. Measured over the full Netlib set:

- simplex pivots reduced (the exact ratio is being re-measured with one variable held fixed, see Section 11; the range across runs is 0.51×–0.77×)
- `degen3` 89,640 → 25,027, `d2q06c` 25,012 → 3,952, `czprob` 5,261 → 1,130
- handed its own optimum, it returns in **0 iterations**

The status column matters more than the ratio. Instances that returned **no answer at all** and now return the right one: `degen3`, `stocfor2`, `woodw`, `scsd8`, `wood1p`, `modszk1`.

The one instance where crossover costs pivots is `scsd6`, and the mechanism is understood: the seeded basis is near-optimal but *primal infeasible*, so the simplex has a phase one to run that a cold start does not. Starting the dual simplex from it instead was tried and mostly falls back to the primal, helping only `grow22` (1,134 → 461).

13.3 Presolve

Eleven reductions: empty/singleton row, empty/fixed column, forcing and redundant rows, doubleton equations with an implied-free guard, free column singletons, dual fixing by lock counting, duplicate rows, bound tightening, coefficient tightening (off).

Removes about **19% of rows and 12% of columns** on Netlib without changing an answer, and recovers dual values as well as primal ones. Reductions that cannot invert their dual exactly carry a `dual_is_exact()` flag and the solver says so rather than returning a shadow price nobody can stand behind.

For MILP it is **off by default**, and that is a measurement rather than an oversight — it is integrality-aware and changes no answer, but it changes the tree in both directions and by a lot: `p0201` 1,291 nodes → 205, `gt2` 500 → 8,320.

13.4 QP

ADMM in the OSQP formulation. The per-iteration KKT system is **quasi-definite**, so by Vanderbei's theorem *any* symmetric permutation of it factorises — which means the ordering can be chosen purely for sparsity, computed once, and reused for every iteration with no numerical pivoting. That is the entire reason the method is fast, and it is why the σI regularisation exists at all.

Direct $LDL^\top$ measured **1.52×** faster than conjugate gradients on the same system.

35 of the 40 smallest Maros–Meszaros instances solve. Five do not: `PRIMALC1/2/5/8` and `QPCBOEI2` — four of them one family, whose `DUALC` counterparts all solve.

The mechanism is diagnosed. Adaptive ρ drives it from 10^{-1} to 1.9×10^{-5} within five hundred iterations; once the weaker primal step lets the primal residual grow to match the dual one, the ratio sits near one, no update passes the threshold gate, and ρ stays where it was left. **The gate that stops it thrashing also stops it recovering.** With ρ held fixed the same instance converges in 7,000 iterations to the published optimum.

Three fixes have failed and are recorded so they are not retried:

1. Limiting ρ's drift — monotonically worse (35/40 unlimited, 33 at a factor of 10^2, 30 at 10).
2. Relaxing the gate after a long stall — no change at any setting.
3. Normalising residuals by term scale rather than by tolerance, which is what OSQP does — **withdrawn on paper**: write the ratio out and the relative tolerance cancels, so the two rules agree whenever the absolute term is negligible, and on `PRIMALC1`, whose primal residual is 74, it thoroughly is. The two rules compute the same number on the instance that fails.

13.5 GPU

Measured **2.70×–7.09×** over the same algorithm on CPU (Tesla T4). Device- resident vectors, fused reductions, adaptive SpMV row-width selection.

Two things known and unfixed:

- **On one instance 79% of solve time is outside the kernels.** The convergence check still runs on the host, forcing a device→host copy each time it runs. Half the fix is written.
- `cudaMalloc` does **not** zero its memory while the CPU allocator value-initialises. That hid a missing upload of the dual iterate for a long time — harmless while everything started at zero, and silently fatal for any warm start on the device.

13.6 MILP

The weakest component, by a wide margin, and the one the problem statement cares most about.

What is in the tree: pseudocost branching with reliability (strong branching until a pseudocost is trusted, capped at depth 10), cover / MIR / Gomory separation with a per-instance keep-or-drop rule based on whether the cuts moved the root bound, reduced-cost fixing, bound propagation into

children, a rounding heuristic, a fix-and-propagate dive, a feasibility pump, and a crossover-seeded root relaxation.

Measured effects of individual pieces:

	before	after
reduced-cost fixing (`gt2`)	7,901 nodes	1,167
node propagation (`flugpl`)	28,917 nodes	477
reliability branching, depth-capped (`gt2`)	783 nodes	194
basis carried through cut rounds (`gt2`)	500 nodes	151
basis carried through cut rounds (`p0201`)	1,291 nodes	831

On the *seven small instances* the defaults were tuned against, four prove optimality and on the other three the **solution** is already the published optimum — what is missing there is the proof, not the answer.

On the wider MIPLIB 2017 set at a fifteen-second limit, most instances finish with no feasible solution at all — seven of the first thirteen. A tree with no incumbent has nothing to prune against and explores blind. That is the honest state of this component and it is why it is the first of the five work streams in Part V.

Three infrastructure problems found while measuring that, all now fixed:

- **The time limit only bounded the node loop.** The root relaxation and eight rounds of cut separation ran unbounded. `10teams` asked for 15 s and took **103.8 s**; four of the first fifteen instances overran.
- **Cut rounds consumed the whole budget legally.** `berlin` solved eleven relaxations for 75,098 simplex iterations and then explored **zero** nodes. Cuts now get a slice of the budget rather than the run.
- **Every cut round rebuilt its probe from nothing** — and each round does two solves, so eight rounds was sixteen cold solves for one root. Adding cuts does not invalidate a basis: the new rows arrive with their own slacks, and a slack basic in its own row is what keeps the extended basis nonsingular.

14. Self-assessment against HiGHS

HiGHS is the realistic open-source bar. Against it:

component	~score	reasoning
MPS reader	95	at parity, and faster
Infrastructure	72	good tests, no CI, no packaging
First-order LP	62	strongest algorithmic piece; no multithreading
GPU	55	real 2.7–7×, not cuPDLP-C level

component	~score	reasoning
Simplex	55	77/88 correct and none wrong, plus crossover. No Forrest–Tomlin, no bound-flipping ratio test, no hypersparsity
QP	50	OSQP's core is here; no AMD ordering
Presolve	38	11 reductions; HiGHS/PaPILO have ~25
MILP	**30**	**the weak leg**

Weighted, about **60/100**.

And one number that is not on that table: zero threads.

```
grep -rn "std::thread\|<thread>\|omp parallel\|std::async" src/ app/
```

returns nothing. Every benchmark in the repository is single-threaded on both sides, which is a fair comparison and an honest one — and it also means an entire multiplier is unused.

15. Where to be judged

Mittelmann's LPfeas benchmark is where the codes this method comes from are actually scored. Over 65 problems:

code	shifted geomean (s)	solved
cuOpt 26.08	13.7	62
COPT 8.0.0	25.3	**65**
cuPDLPx 0.3.0	28.5	57
HiGHS 1.15.0	256	55
OR-Tools PDLP 9.10	421	50

That last row is worth sitting with. **OR-Tools' PDLP is the same algorithm family implemented here, and it solves the fewest of any code except KNITRO.** The method is not automatically good; the implementation is most of it.

Eight of that benchmark's forty public instances are already in the repository. Reporting solved-or-not at 10^{-6} against that published table is a comparison anyone can check, and it is worth more than any score computed against a rubric we wrote ourselves.

Part V — What happens next

16. Five work streams

The parts furthest behind, split so they can be worked in parallel on separate branches. Each carries its own brief: the problem with the evidence for it, what has already been tried and failed so nobody repeats it, and directions to research rather than a solution to implement.

Stream 1 — MILP search (branch `milp-search`)

Score 30, realistic ceiling ~50. Biggest gap and heaviest weight, because refinery scheduling *is* a MILP.

The target is the number in Section 13.6: most MIPLIB instances finish with no incumbent. Missing: conflict analysis, restarts, improvement heuristics that need an incumbent (RINS, local branching, sub-MIPs), cut pool management with ageing — right now a cut enters the matrix and stays forever, making every node below it more expensive — clique tables, symmetry detection.

Known dead ends recorded in the brief: Gomory in every round (the root bound on `gt2` runs past the optimum and comes back down, and a bound that falls after a cut is not a bound), strong branching without a depth cap (`gt2` 783 → 2,225 nodes).

Stream 2 — Presolve depth (branch `presolve-depth`)

Score 38, realistic ceiling ~55. Eleven reductions against HiGHS/PaPILO's twenty-five.

The headline candidate is **probing** — fix a binary to 0 and to 1, propagate each, keep what both agree on. Usually the single highest-value MILP reduction and it is not here. Then clique extraction, dominated columns, implied integers, aggregation.

Its score weight understates it: presolve makes every downstream component faster for free, so it multiplies rather than adds.

The danger is also the highest here. Every reduction is a claim that certain solutions can be discarded, and one wrong claim discards the answer — the first wrong answer this project ever produced came from presolve, where a 10^{-9} bound padding manufactured a forcing row and a feasible model was declared infeasible.

Stream 3 — Parallelism (branch `parallel`)

Currently zero threads. Candidates, in rough order of independence: cut separation (per row), strong branching probes (independent bounded LPs), simplex pricing (a pass over columns), the first-order CPU inner loop, node processing (hardest, shared incumbent).

Determinism is the requirement, not a caveat. A parallel MILP returning a different answer — or the same answer after a different node count — on each run cannot be demoed, benchmarked, or debugged.

One warning from the existing evidence: a sampling profile of `degen3`, the slowest Netlib instance, put 67% of samples in one place and the LU factorisation at **11 samples out of ~4,500**. Profile before choosing; intuition has been wrong here before.

Stream 4 — GPU (branch `gpu`)

Score 55, realistic ceiling ~70. The problem statement's own premise, so it carries more weight in front of a judge than its share of the code.

Two concrete items: finish moving the convergence check onto the device (the 79% figure in Section 13.5 is a pending task, not a research question), and re-verify the whole path on hardware, since nothing built recently has run on a real GPU and the CPU path hides a whole class of device-only bug.

Practical constraint: the development machine is Apple Silicon and has no NVIDIA GPU. Everything is built and run on rented hardware (`docs/KAGGLE.md`, `scripts/package_for_kaggle.sh`). Plan for a slow edit–test loop and batch the work.

Stream 5 — QP (branch `qp`)

Score 50, realistic ceiling ~62. Five failures, three fixes already tried and recorded.

Untried and worth a look: a hard restart — ρ reset to its initial value with the factorisation redone — which is structurally different from the three failures. Also AMD ordering for the $LDL^\top$ (there is none; measure the fill before assuming it matters), iterative refinement on the KKT solve, and extending the test set beyond the 40 *smallest* instances, since 35/40 on the smallest may be flattering.

And a legitimate outcome: if ADMM cannot be fixed for this family, saying so with evidence and scoping what a barrier method would cost is a real deliverable.

What is held outside the five

The simplex core, crossover, the ratio test and the Netlib verification harness are being worked separately. Streams 1–5 leave `src/sankhya/simplex.cpp`, `src/sankhya/crossover.*` and `bench/verify_simplex.py` alone.

17. What the five are worth

Estimates, with the reasoning attached rather than hidden.

stream	component now	realistic	on the composite
MILP	30	50	**+4 to +5.5**
Presolve	38	55	+2.5 to +3.5
Parallelism	—	—	+3 to +5
GPU	55	70	+2 to +3.5
QP	50	62	+1 to +1.6
		total	**+12 to +18**

So roughly **60 → 72–78** if all five land, and **~70–73** if three of five do, which is the more realistic expectation — each stream starts cold, and the MILP one is genuinely research-heavy.

Two things could make this go backwards. Parallelism that breaks determinism is worse than no parallelism. And any stream that introduces a wrong answer costs more than all five gain together — in front of a judge, one wrong number invalidates every other number on the slide.

18. Honest open questions

Things that are genuinely not settled, listed so nobody assumes they are.

The crossover ratio. Two changes landed close together — the ratio-test fix and a change to how crossover ranks candidates — and the full-set pivot ratio moved from 0.51× to 0.77× with no way to attribute it. A single-variable re-measurement is running. This is the "change one thing at a time" rule being violated and then paid for.

Whether `scsd6`'s crossover regression is fixable. The mechanism is known (near-optimal but primal-infeasible seed → phase one). Neither the dual simplex nor a tighter seed tolerance is a clean fix.

Whether MILP presolve should be on. It changes no answer and changes the tree by an order of magnitude in both directions. That is a per-instance decision nothing in the code is yet equipped to make.

Whether `cycle` is solvable here at all. It is genuinely ill-conditioned. It currently fails visibly, which is correct behaviour, but "correct behaviour" is not the same as solving it.

Whether the GPU numbers still hold. They were measured before a substantial amount of recent work and have not been re-run on hardware.

19. Getting set up

```
git clone https://github.com/team-vertexx/sankhya && cd sankhya
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j8
for t in build/tests/test_*; do "$t"; done          # 12 suites, all should pass

python3 scripts/fetch_netlib.py                      # 88 LP instances
python3 scripts/fetch_miplib.py                      # 103 MILP instances

build/sankhya simplex data/netlib/25fv47.mps --presolve
build/sankhya simplex data/netlib/degen3.mps --crossover
build/sankhya solve   data/refinery/refinery.mps --presolve
build/sankhya milp    data/miplib/flugpl.mps --time-limit=30
```

Documents in the repository, in reading order:

file	what
`README.md`	the short version
`docs/ARCHITECTURE.md`	component design, 403 lines
`docs/COMPLETE_GUIDE.md`	the mathematics of every method from first principles, 41 pages
`docs/RESULTS.md`	every measurement with its command
`docs/ONBOARDING.md`	practical setup, and the traps in Section 11
`docs/ROADMAP.md`	what is open and why
`PLAN.md`	the original build plan, plus a Part II revisiting it

The conventions that matter. Every default in this codebase is a measurement — if a constant changes, the comment beside it says what was measured and on what. Options that were implemented and turned out not to pay are kept in the source, switched off, with their numbers beside them. There are several of those and they are as useful as the features.

20. The thing worth taking away

The algorithms were not the hard part. They are in papers and they work.

What took the time was finding out **when the solver was quietly wrong** — and building the things that make that visible.

A slow solver tells you it is slow. A wrong one tells you nothing: it hands back a confident number with a matching dual bound and no complaint. On `fiber` it was wrong by 60% and looked completely healthy. On `grow22` it reported optimal at a point missing the constraints by 1.8×10^9. On `modszk1` it declared a bounded model unbounded.

None of those was found by a test that was written for it. They were found by checking answers against published values, by two independent methods disagreeing about the same problem, and by a third solver settling which of the two was right.

Every self-check in Part III exists because something got through.